Verifiability Is a Spectrum: A Single-Auditor, Artifact-Only Audit of Google's Sealed-Hardware Federated Learning Pipeline

Anonymous Author(s)

Abstract

Google's federated learning (FL) system—which trains inside Trusted Execution Environments (TEEs) and is deployed in production for Gboard next-word models—claims that encrypted uploads are "decrypted only inside sealed hardware running publicly listed code" and that its central differential-privacy guarantees are externally verifiable. We ask what "externally verifiable" means operationally: for each externally-checkable claim, can an outsider with no device, no credentials, and no Google contact actually verify it from public artifacts? We develop a seven-dimension audit framework (attestation binding, code transparency, policy expressiveness, key lifecycle, firmware/boot chain, side-channel surface, operator-insider residual), score each checkable claim as *verified*, *partially checkable*, or *trust-required*, and apply it in a worked audit grounded in live public artifacts. Of the 27 rows in our condensed matrix—24 scoring vendor assertions about the system, two scoring disclosures whose content is the statement itself, and one recording a scope limit rather than a claim—only 2 of the 24 vendor-assertion rows score *verified*; the two disclosure rows also score *verified*, bringing the condensed totals to 4 *verified*, 10 *partially checkable* (one a negative finding) and 13 *trust-required*—all of them a single auditor's scores (§3 reports the reliability exercises and their limits). The *verified* count is 3–5 across the envelope of row constructions and strict-direction scoring variants of the published matrix at its post-amendment state (4 under ours—the pre-amendment snapshot reads 3/10/12, §5; one of the four amendment-raised, a replication awaiting blinded repetition, §4); the crossing is §5's, and the full 29-row matrix is Appendix B. None of the four *verified* scores attests runtime behaviour: two are executed artifact verifications (the transparency-log chain for the documented exemplar, verified end to end including the log's own signatures; and the public source and build recipe), and two are verified statements whose claims are the statements themselves (a stated freshness window; an acknowledged side-channel limitation). We replicated the guide's documented worked example end to end—transparency log, endorsement, binary, build provenance (at digest equality), source—recomputing every hash. The chain holds for the exemplar: log membership included, and the checkpoint, timestamp, and endorsement signatures verify under their published keys. A later census of all 115 log entries sharing the exemplar's signing key byte-verified 110 endorsed payloads (the 5 earliest no longer served). Extended to every published endorsement key we could reconstruct—19 keys, 509,569 entries, as returned by the v1 index endpoint—the census shows what the published keys have endorsed *is* enumerable. Whether the vendor's key inventory is itself complete is a separate, trust-required claim; whether the index returns are complete against the log's tree is a third (row C3c); and payload readability is bounded by retention. Twenty SLSA v1 provenance statements from twenty distinct source commits assert the identical subject digest for the KMS binary—consistent with toolchain determinism, but also with build caching: input-sensitivity is not established (14 of the 20 commits touch neither the KMS tree nor the shared module file; §4.1). The underlying reproducibility claim now has an executed check: the project's own CI build invocation, run by us on a hosted cloud environment at the exemplar-era commit, reproduced the endorsed digest bit-for-bit for that one binary. (Each of these results arrived by a flagged route—post-October amendments, an exploratory rescue—detailed in §4.) The store's retention has already expired the audit guide's own worked example; a substantial share of older endorsed payloads is already unserved; and the published key inventory on which enumeration rests shows drift—two guide fingerprints match no current key. Live access-policy endorsements are retrievable in bulk; recursive wire-level decoding of every retrieved policy artifact found complete Python workload programs embedded (base64-encoded) in five distinct artifacts of the sampled retrieved set—including the Gboard training programs, each carrying its `_MAX_ZCDP` guard (57 artifacts; the other 52 are binary configurations, §4.1): the embedded-program mechanism is verified as documented and observed on live policies; the five are an existence count over the retrieved set, not a prevalence rate (§5). What survives is the residual-trust enumeration itself, and a checklist from a single-auditor case study. Whether the methodology transfers to other sealed-hardware training pipelines is a hypothesis for other audits to test.

1 Introduction

"Auditable" is a spectrum, not a switch. When a production machine-learning system claims its training pipeline is externally verifiable, the claim can mean anything from "we published a white paper" to "any outsider can re-derive every checkable assertion from public artifacts, today." For sealed-hardware *training* pipelines, it has not, to our knowledge, been mapped.

In October 2026, Google announced a next-generation federated learning system built on Trusted Execution Environments (AMD Secure Encrypted Virtualization–Secure Nested Paging (AMD SEV-SNP), Intel TDX—Intel Trust Domain Extensions) that moves model training server-side into TEEs [1]. Devices upload training data encrypted under keys held by a TEE-hosted Key Management Service (KMS); uploads are cryptographically bound to access policies naming the exact Python programs and container binaries permitted to process them; policies and binary endorsements are published to the

This work is licensed under the Creative Commons Attribution 4.0 International License. To view a copy of this license visit https://creativecommons.org/licenses/by/4.0/ or send a letter to Creative Commons, PO Box 1866, Mountain View, CA 94042, USA.
Proceedings on Privacy Enhancing Technologies 2027(3), 1–21
© 2027 Copyright held by the owner/author(s).
https://doi.org/XXXXXXX.XXXXXXX

public Rekor transparency log; and the KMS and data-processing binaries are claimed reproducibly buildable from open source. The system is in production training Gboard English and Japanese next-word models, with substantially faster training on a smaller privacy budget than the prior system in the live A/B test reported in §7 of [1].

The paper's verifiability claims name logs, keys, hashes, and build procedures—things an outsider can check. So we put the question in falsifiable form. Could an independent party, with no participating device, no Google credentials, and no insider contact, verify each externally-checkable claim from public artifacts, using documented procedures, as of a stated date? Where the answer is yes, we show the receipts: hashes, log indices, timestamps. Where it is no, we name the failing link—and what would fix it.

One clarification before the scores. *Verified* here means the public artifacts are mutually consistent—hashes match, log entries resolve, provenance chains close. It does not mean the artifacts describe what actually runs on Google's servers. A malicious operator serving self-consistent artifacts defeats every outsider procedure; we score that residual explicitly—and the question's own terms set that boundary: *verified* certifies consistency of the public artifacts, the only thing verification *from public artifacts* can certify; claims needing the further step to the running system score T, never V.

Contributions. (1) A seven-dimension audit framework—proposed here and not yet validated beyond this case study—for sealed-hardware training pipelines with a three-way scoring rubric (*verified* / *partially checkable* / *trust-required*, defined in §3) and a threat-model-driven scoring of each dimension. (2) A worked, artifact-only audit of Google's deployment—including the wire-format decode that found complete Python training programs embedded in five live policy artifacts (§4.1)—grounded in live artifacts pulled October 4–8, 2026, with re-probes on 2026-10-15, amendment experiments on 2026-11-04, and an endpoint re-probe on 2026-11-05 (§4). To our knowledge—and within the literature a scoping search surveyed (§8)—this appears to be the first outsider verification attempt for a production sealed-hardware *training* pipeline; the nearest misses are Google's own first-party audit guide and the WiSec'26 PCC analysis, which is inference-only reverse engineering. (3) A document-based juxtaposition of Apple Private Cloud Compute on the same dimensions—from published documentation, not executed procedures, and not a scoring. (4) An explicit residual-trust enumeration: the set of claims that cannot be verified without Google's cooperation, which we argue *is* the audit's main deliverable.

Scope and non-goals. We audit *verifiability claims*, not the underlying ML or DP mathematics, and not TEE hardware security. We do not attack the system, access non-public data, or interact with Google. All procedures use public endpoints and are reproducible from our artifact package.

2 Background: The System Under Audit

We summarize only what the audit needs; the system itself is described in Daly et al. [1] (v4, September 30, 2026) and a companion blog post [2].

Architecture. Client devices (here: Gboard, Google's keyboard app, so training examples derive from users' typing) encrypt training examples under a KMS-issued public key and upload them to intermediate storage, binding each upload to an *access policy*. The KMS is a cluster of TEEs ("sealed hardware" throughout means TEEs—hardware-isolated compute whose software measurements are cryptographically reported) running the RAFT consensus protocol holding a rollback-protected key–value store. Later, a server-side "root" TEE executes a workload author's Python program (signature `trusted_program(external_handle)`) and may delegate parallel work to worker TEEs over a Noise-protocol channel, with the root validating worker attestations—hardware-signed reports stating the cryptographic measurements (hashes) of the software a TEE runs. The KMS releases decryption keys only to TEEs whose attested measurements match the policy's *reference values* (four stack layers: Stage0 firmware digests + SEV-SNP security version number (SVN) thresholds; kernel/initramfs digests + boot cmdline; base OS image digests; container binary SHA-256).

Verifiability design. Before uploading, devices validate that (1) the KMS's TEE software and (2) the access policy are recorded in a public transparency log (Rekor [4]), including log-inclusion proofs. For Python workloads, the policy *embeds the program text* plus root/worker binary hashes, so "only that specific Python program, executed by those specific binaries, can access the uploaded data" [1]. Policies are published to Rekor; "external auditors can view how the program establishes DP guarantees" (e.g., a program-level `_MAX_ZCDP` guard, a zero-concentrated-DP bound). One qualifier belongs here: model architectures and preprocessing protos are sideloaded at runtime and deliberately *not* in the log, so the pinning covers the named program, not everything it loads; the assurance that all privacy-relevant logic is nonetheless pinned is itself trust-required (§5). The paper states the policy-listed binaries are "reproducibly buildable TEE containers built from OSS source code," and points to the `google-parfait/confidential-federated-compute` repository, whose docs give instructions for mapping log entries to source commits [3].

Threat model (ours). We evaluate each claim's residual against six adversaries (labelled Adv. 1–6, to keep them distinct from the claim rows A1–G4): **Adv. 1** honest-but-curious pipeline operator; **Adv. 2** malicious (active) operator; **Adv. 3** holder of a compromised endorsement key; **Adv. 4** TEE hardware/firmware attacker; **Adv. 5** malicious workload author; **Adv. 6** network observer. Capabilities in brief: Adv. 3 holds one endorsement key; Adv. 4 operates beneath the software measurements (hardware/firmware); Adv. 5 authors workloads but not infrastructure; Adv. 6 observes network traffic only. Adv. 4 and Adv. 6 are not scored claim-by-claim: no procedure in our persona engages them, so they are carried as named residuals (Table 5)—a scope boundary, not a failed check. The operator is *not* assumed honest. We additionally treat operator↔log-operator collusion as an explicit *assumption* of the threat model, not as a finding: Rekor is operated by Sigstore/OpenSSF, and Google co-created Sigstore (with Red Hat's Open Source Security Team) [22]—a founding association that establishes neither current operational control nor shared signing authority, and we allege neither. Under the assumption, the operative question is what a colluding log could hide and what would detect it: signed checkpoints and consistency proofs bound equivocation for monitors who retain them, but Rekor offers no cross-witness protocol (the failure mode LogPicker [21] makes detectable for CT logs), so split-view detection rests on monitors comparing checkpoints out of band.

3 The Audit Framework

Our framework has three parts.

Dimensions. Seven, spanning the chain from silicon to operator: (1) *attestation binding* (device→KMS→TEE chain of trust); (2) *code transparency* (source availability, reproducible builds, provenance); (3) *policy expressiveness* (what the access policy pins down, and whether the "complete set of workloads" is enumerable); (4) *key lifecycle* (endorsement-key publication, rotation, custody); (5) *firmware/boot chain* (reference values, hardware roots); (6) *side-channel surface*; (7) *operator-insider residual* (what a privileged insider could do or hide). Coverage is of claims an outsider can state a procedure for: one obligation the guarantee composes from—DP output gating with atomic release—has no checkable claim or artifact in the deployment, hence no row; Table 7 marks it not examinable.

Scoring. Each checkable claim is scored **V** (verified: we executed the check end-to-end on public artifacts), **P** (partially checkable: sound in principle, degraded or blocked in current infrastructure), or **T** (trust-required: no outsider procedure exists). Scores are evidence-anchored: every V cites the artifact, hash, and date; every P names the broken link; every T names the trusted party. Terminology, used consistently below: *verified* means executed and recomputed by us; *authenticated* means a signature check passed; *observed* means returned by a public endpoint and recorded, without independent recomputation. One vocabulary discipline governs every count in this paper: a *row* carries exactly one score letter, a *table* aggregates by counting rows carrying a given letter, and a count printed anywhere below is a count of rows—never of artifacts, payloads, policies, or endorsements, which are counted only in §4, always with their population named. Four rules keep the rubric honest. First, disclosure: a row scores V on a vendor statement only when the claim audited *is* the disclosure (e.g. that a limitation is acknowledged); where the claim is a fact and the only support is the vendor's statement of it, the row scores T—a verified statement does not verify a fact. A companion precedent: where the claim audited is that an artifact is public (source, build recipe, reference values), fetching and inspecting the artifact *is* the execution, and the row can score V (row B1). Second, scores grade *checkability*, not truth: where an executed check contradicted the claim as worded (row C5), the row's basis states the contradiction, and that P score should be read as a negative finding, not reassurance. Third, a row whose procedure exists but was not executed is scored P and flagged "unexecuted" in its basis; it is never scored V (nothing was executed) or T (a procedure exists). The rule governs claim rows; a scope-limit record, which scores no vendor claim, is flagged on its own terms (row G4, Table 1). Fourth, a line for code inspection, which several rows turn on: a procedure counts toward P only if the scored persona could execute it on public artifacts *and* its execution would decide the claim as worded. Static inspection of the public client code path meets that test for the published client implementation (row A2), and we executed it (2026-11-04): the published client library's attestation verifier checks the upload-key endorsement against the KMS verifying keys and the pipeline configuration against the policy verifying keys, verifies Rekor inclusion proofs by root-hash recomputation, and cross-checks the policy digest carried in the key endorsement. What that decides is the public client implementation (the code runs on the verifier's side of the boundary, and what the client does is the claim); that production devices run it is not observable, and it scores separately as row A2b (T)—the corpus rule below, applied to A2 at this revision on the same footing as the enforcement halves of C3 and C4 (rows C3b, C4b): bundled statements are split wherever their parts score differently. Rows C1 and C2 rest on the same line from the other side: inspecting the published protocol definition and the guides' worked policy decides the pinning and budget claims as worded for the published artifacts, so they score P by inspection although no live policy byte was executed. They are the weakest P scores in the matrix: inspection of published artifacts is the evidence class that failed on row B4, where the decisive content stayed invisible until wire-level decoding superseded the scan (§4). We keep the line because the claims are about the published artifacts themselves—what the definition and worked example pin down is decided by inspecting them—and B4's failure was of the scan applied to retrieved bytes, not of inspection: the same bytes, wire-decoded, decided B4. It does not meet the test for claims about runtime behaviour inside the operator's TEEs (rows A3, C4b, D3, G3): inspecting server code shows what the code says, not that the running system does it, and the running binary's identity is itself reachable only conditionally, by a participating device under row E2's hardware assumptions (§5); those rows score T, although their code is equally public. The full 29-row matrix is in Appendix B. We know of no existing outsider-audit rubric for sealed-hardware *training* pipelines to benchmark these dimensions against (existing TEE evaluation guidance targets hardware assurance, not claim-by-claim checkability); the seven dimensions are our proposal, derived from the system's own claim chain—each layer at which the vendor asks an outsider to check something becomes one dimension.

Claim corpus and extraction. The scored claims are extracted from a fixed corpus: Daly et al. [1] (v4, 2026-09-30), the companion blog post [2], and the repository documentation as fetched 2026-10-08 (the endorsement and attestation inspection guides, the KMS design document, and the reference-values files). A statement enters the matrix iff it asserts a property an outsider is invited to check from public artifacts; bundled statements are split whenever their parts score differently (the C3/C3b, C4/C4b, C3/C3c, and A2/A2b splits are instances of this rule, not exceptions). Each row is assigned to the dimension naming the chain layer at which its check executes; firmware is separate because its artifacts and trusted parties differ from the runtime rows'. The verbatim claim texts and per-row evidence are in the artifact's `AUDIT_MATRIX.md`; Appendix B prints the matrix, the full claim wordings as scored (Table 8), and the second-scorer exercise's agreement table (Table 9).

Verifier classes. "Outsider" is not one vantage point. We distinguish (i) the unauthenticated public observer—no device, no credentials, no operator contact; this is the persona our scores are reported for; (ii) the independent researcher with a commodity cloud account, who gets bulk-data routes (the BigQuery dataset) and rebuild-class compute—used for the §4.3 rebuild, and flagged where used; (iii) the participating device owner, who can additionally validate attestations device-side; and (iv) the cooperating external auditor, whose vendor-facilitated access places them outside "outsider" as we use the term. Where a row's answer changes by class, the basis says so; a T score means no procedure for class

(i), not that no class could decide the claim. The framework is a proposal, not a validated instrument: the original scoring is by a single auditor; a second-scorer plausibility check on the condensed rows and an automated artifact-level re-scoring of the matrix as executed are reported below; no test–retest exercise has been done, and we present the framework as a case-study checklist whose dimensions other audits should confirm, drop, or extend. A limited reliability check on the *scores* (2026-11-04): a second scorer, blind to the first scoring, re-scored all 23 condensed rows as then constituted (the matrix has since grown; the added rows were not re-scored) from the evidence presented here and agreed on 21 of 23. Both disagreements argued our scores were too generous, and both were adopted at the time: the per-round-upload-use row names the disclosure that was actually verified (matching its full-matrix twin), and enumeration was rescored T on the evidence then presented, applying the §7 predicate strictly; on the corrected record it stands at P, for the reasons §4.4 states (the scorer judged the evidence as presented, which contained the first pass's mistaken premise about the key index). Chance-corrected agreement is Cohen's $\kappa = 0.85$ ($n = 23$; the 3×3 table and both itemized disagreements are in the artifact's second-scorer record). κ describes agreement with the scores as held at the time of the exercise—one of them (enumeration) has since changed on corrected evidence, so κ does not characterize the published matrix. The published matrices have also since gained rows (the C3 and C4 splits, §4.4 and §5, and at this revision the A2 split and the index-completeness split C3c—both outside both reliability exercises); the exercise's counts describe the matrices as then constituted. One limit of that exercise stands: the second scorer worked from the evidence *presented in this paper*, not from the raw artifacts, so it tests whether our presentation supports our scores—a plausibility check on a superseded constitution of the matrix, and the only human second-scoring in our record.

On 2026-11-04 an automated scorer within our own tooling—a separate agent in the same pipeline, blind to the published scores, not a human auditor—re-derived all 27 rows of the audit matrix as executed—the matrix recorded in the artifact's `AUDIT_MATRIX.md`, which is the published full matrix without the two revision-stage split rows (A2b, C3c)—from the artifact package (the sealed artifact, the reproduction packages, and the experiment records) under the rubric above. It agreed with the published score on all 27 rows and changed none. Four rows where a competent scorer could land elsewhere (A3, A4, B2, G4) were adjudicated on the rubric's own precedents; the score stood in each (adjudications in the artifact's re-scoring record). We report the raw outcome only, with no chance-corrected statistic: the scorer shares our pipeline and rubric, so the exercise evidences that the as-executed matrix is *reproducible from its own artifact-level evidence* under its own rubric—not independent confirmation; and that matrix had already absorbed the corrections above, so it validates the as-executed matrix's reproducibility, not the original scoring act's reliability. No human second scoring of the final matrix from the raw artifacts exists: every score in the published matrix is a single auditor's, and the audit is a single-auditor case study throughout. All quantitative claims are single measurements or census counts over the audited artifacts; we make no inferential comparisons, and rates derived from single measurements are reported as order-of-magnitude where the text says so.

Comparator design. A single-case audit should indicate whether its dimensions separate design choices from verification gaps or merely relabel one system's properties. Our only instrument for that here is a juxtaposition: a second production confidential-computing deployment—Apple Private Cloud Compute (PCC)—set against the same dimensions, from documentation (§6). PCC is inference-only and architecturally different. The juxtaposition is illustrative, not a validation: document-based and unscored, it can show only that the dimensions return a different mixture of design choice and verification gap for a second system on paper, not that they discriminate in general; whether they do is part of what other audits must test. One asymmetry: the Google scoring above rests on executed procedures, the PCC juxtaposition on documents alone.

4 Worked Audit

The October procedures below were executed October 4–8, 2026, from an unprivileged network vantage point (no device, no credentials): verifier class (i) of §3. The single exception is the §4.3 rebuild (2026-11-04), which used class (ii), a commodity cloud account, as flagged there; the further 2026-11-04 additions are listed next. Scripts and raw outputs are in the artifact package; every hash below was recomputed. **Audit timeline and amendments.** Re-probes followed on 2026-10-15 (inclusion-proof verification, key-index endpoint), and an endpoint re-probe on 2026-11-05 (Appendix C). On 2026-11-04, under amendments frozen before any of their runs, we added (we flag the pattern ourselves: both amendments ultimately raised scores, and they were written after the October results were known—frozen before their runs, but not blind to the audit's direction; readers should weight the two raised scores accordingly): the signing-key and KMS digest censuses, an independent re-implementation of the inclusion-proof verification, the provenance-commit classification, and the measured retrieval-rate experiment (§4.1, §4.4); a second amendment the same day added the all-keys census, the exemplar's signature authentication (checkpoint, signed timestamp, endorsement signature), and the policy-payload classification (§4.1). Two review-stage additions later the same day—a wire-format decode of every retrieved policy artifact (superseding the string-scan classification for row B4, §4.1) and the executed static inspection behind row A2 (§3)—are flagged where they occur. Confirmatory status, stated plainly: the amendment experiments were frozen before their runs and are confirmatory for the procedures they specify; the §4.3 build success is not—the frozen command failed at the era tree and the working invocation was found in the project's CI workflow mid-audit, so that result is an exploratory rescue, reported as such in §4.3. One deviation is disclosed: the signing-key census was extended from the frozen first-10 sample to all 115 returned entries; both results are reported. The frozen amendment text is in the artifact package (`AMENDMENT-2026-11-04.md`); the experiments changed two letter scores (A1 to V on the authenticated chain, §4.1; C3 to P on the corrected enumeration route, §4.4) and the stated basis of two further rows (B2, B4); the review-stage additions changed no letter score—they executed A2's procedure, and replaced B4's non-detection basis with an observation in the vendor's favour (we flag that direction too); every other October conclusion stands.

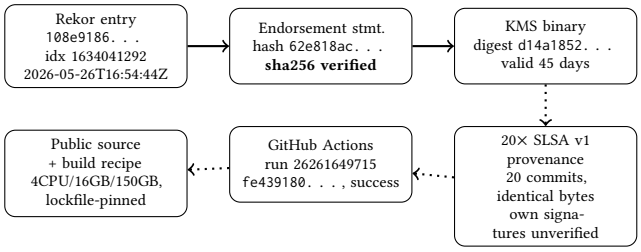


Figure 1: The outsider-verifiable chain for the KMS binary. Solid edges were re-derived by us from public endpoints for the guide's documented exemplar (hash recomputation; the endorsement statement's own signature verified, §4.1); the from-scratch binary rebuild was executed with the project's own CI invocation (§4.3). Dotted edges rest on provenance statements fetched from the GitHub Attestation API whose own signatures we did not independently verify (§4.1).

4.1 Log → endorsement → binary → source

Figure 1 shows the chain we verified end-to-end; we recomputed every edge below on 2026-10-04/08, not copied from documentation, and executed the from-scratch rebuild on 2026-11-04 (§4.3). Terms, at first use: *hash-verified* means we recomputed the SHA-256 ourselves and it matched; a *live* artifact is one the store still served on the audit date; the *wire format* is an artifact's serialized protobuf field stream, readable field by field without the vendor's schema; a *complete program* is embedded text that parses as a full Python program—imports, definitions, and a training entry point—not a fragment or a marker string.

We walked the documented verification chain for the KMS binary. Google's audit guide [3] gives a worked example: Rekor entry UUID `108e9186. . .` (full UUID in Appendix C) We fetched it from `rekor.sigstore.dev/api/v1/log/entries/{uuid}`: log index **1634041292**, integrated time **2026-05-26T16:54:44Z**. The entry body names endorsed hash `62e818ac. . .`. We fetched `https://federatedcompute-pa.googleapis.com/data/transparency/sha2-256:62e818ac. . .` and computed SHA-256 over the returned bytes: **it matches the endorsed hash exactly**. The payload is an in-toto (a supply-chain attestation format)/Oak endorsement statement whose subject is the endorsed KMS artifact digest (the application bundle; §4.3) `d14a1852. . .`, valid 2026-05-26 through 2026-07-10 (a 45-day difference—46 calendar days inclusive; day counts in this paper are differences). Three notions must not be conflated here: *live validity* (after 45 days the endorsement authorizes nothing new), *historical verifiability* (the signed statement remains checkable while its bytes and keys are served—we verified this expired exemplar's signatures on 2026-11-04, months after expiry), and *availability* (whether the store still serves the bytes at all—the binding constraint, §4.5).

Provenance: querying the GitHub Attestation API for that digest returned **20 SLSA v1 provenance statements from 20 distinct source commits** (spanning 2026-05-22 to 2026-06-03), every one with subject digest equal to `d14a1852. . .`, builder `github.com/google-parfait/confidential-federated-compute/.github/workflows/reusable_build.yaml` (the statements' own signatures were not independently verified). Verifying them would mean validating each statement's Sigstore bundle—its DSSE signature and Fulcio certificate chain, and the bundle's own Rekor entry—for all twenty; we did not execute that validation, because the provenance observation scored here is the statements' published existence and subject-digest agreement, and their authenticity is carried as residual trust (row B3). The pattern is consistent with toolchain determinism, though also consistent with a stale build cache that never rebuilds. Whether the 20 commits even touched the KMS build inputs is answered by classifying every commit's changed files through the GitHub API (2026-11-04): only **1 of 20** commits modifies files under `kms/`; 5 more touch the root `MODULE.bazel` (shared dependency declarations); **14 of 20 touch neither**. For those 14, an identical KMS digest is expected on the screened paths whether or not the toolchain is input-sensitive, so the 20/20 observation bounds toolchain determinism rather than proving input-sensitive reproducibility; and we did not compute the commits' full dependency closure, so we claim only that the identical digest is unsurprising for those 14 on the screened inputs, not that no build input changed. The referenced Actions run (26261649715, "Build and attest," conclusion `success`, head SHA `fe439180bb24e65ad46e869eddee88f9c9b34875`) and the full build recipe are public; the recipe pins its dependencies to a lockfile (`--lockfile_mode=error`—pinning, not network-hermeticity: our first build attempt failed at dependency download, §4.3), runs on a 4-CPU/16GB/150GB runner, and signs provenance via `actions/attest-build-provenance@v1` to Rekor.

We state the trust assumption explicitly: these 20 statements are issued by Google's own CI infrastructure, so they are evidence against a buggy or non-reproducible toolchain, not against a malicious operator who controls that infrastructure (threat model Adv. 2); the Adv. 2 residual stands, and is scored accordingly in §5. A second scoping note, on the inclusion proof: on 2026-10-15 we executed full RFC-6962 inclusion-proof verification—recomputing the leaf hash from the entry body (it equals the UUID's suffix; the UUID's prefix is the tree ID, 1193050959916656506) and folding the API-returned sibling hashes to the checkpoint root with an implementation validated on synthetic trees. The proof verifies at leaf index 1512137030, while the entry's own `logIndex` field reads 1634041292: different index spaces, not a contradiction—Rekor shards its log, the top-level `logIndex` is the *virtual* index (the leaf index plus preceding shards' tree lengths, 121,904,262 leaves here), and the proof is per-shard, against the entry's own tree checkpoint. We record a correction plainly: our first reading of this pair treated them as one index space and reported an API self-contradiction; Rekor's `VirtualLogIndex` semantics dissolve it, and a second, independent RFC 6962 implementation (2026-11-04, freshly fetched entry and proof) confirms the fold. Log membership for the exemplar is therefore verified, and on 2026-11-04 we closed the remaining authentication gaps: the checkpoint's signature verifies against Rekor's public key (log ID `c0d23d6a. . .`), the entry's signed timestamp verifies over Rekor's canonical entry JSON (a working negative control: the same fields serialized in a non-canonical order do not verify, so the check discriminates the serialization, not merely the presence of a signature), and the endorsement statement's own signature verifies under the endorsement key embedded in the entry—which is identifiably the published

application endorsement key, fingerprint `ea0d1f8f...` (§4.2). A final scoping note: the chain replicates the guide's worked example, so it verifies the published procedure on the vendor's chosen entry (independently sampled entries are taken up next), and it verifies *logging*—that this entry, endorsement, and digest were recorded and authenticate as shown—not deployment.

Beyond the worked example: the signing-key census (2026-11-04). Under a frozen amendment, we ran a census of every Rekor entry sharing the exemplar's entry-signing key, retrieved through the index endpoint of §4.4—115 entries in total. No entry was chosen by us; the population is nonetheless one key's entries, reached through the vendor's worked example; the all-keys census below widens the frame. For each we fetched the endorsed payload from the transparency store and recomputed its SHA-256. **110 of 115 payloads byte-match their endorsed hashes** (48 of them KMS endorsements, the rest ledger and other Oak endorsements); **zero mismatch**; the 5 failures are the earliest entries, whose payloads the store no longer serves (HTTP 400). What this verifies, precisely, is the entry→payload hash link on independent samples—plus, for the 48 KMS endorsements, the payload fields themselves: every one's subject digest and validity window parses, and the 48 carry **24 distinct endorsed KMS artifact digests** (2025-06-04 to 2026-11-03), so the endorsed digest demonstrably changes as the system evolves; the 20/20 identical-digest window above is a 12-day slice of a moving target (the executed rebuild of §4.3 covers exactly one of those twenty commits' trees). That entry→payload link therefore verifies on independent samples wherever the store still serves the payload—and the 5 unserved payloads are themselves an instance of the retention finding (§4.5).

All-keys census (2026-11-04). We then extended the census to every published endorsement key we could reconstruct from the repository's reference values and the guide: 19 keys (6 KMS-layer keys, the application key above, 8 Gboard policy keys, and 4 other apps' policy keys), returning **509,569 entries** in total through the same index endpoint, no credentials required. The counts are the index's returns, whose completeness against the log's tree we did not independently verify (row C3c, §4.4). (A seeded cross-check against the tree followed in the review stage: 24 entries—six per key, drawn by a seeded uniform sample (seed 20261104) from the index returns for four keys (application, Gboard policy 1, Recorder, Private Compute Services)—every inclusion proof folding to its served checkpoint root, every checkpoint signature and signed entry timestamp verified; §5.) Payload checks were exhaustive for every key with at most 120 entries—all KMS-layer keys, the application key, and Gboard policy key 2 (the six remaining Gboard keys returned zero entries)—and sampled for the five keys with more (Gboard key 1, Google App, Pixel Search, Private Compute Services, Recorder)—30 evenly spaced entries per key, 31 for Gboard policy key 1 (the 30 plus the guide's named entry); the per-key table is Appendix Table 3. Of 428 payload fetches attempted in this extension, **320 byte-match, zero mismatch**, and 108 returned HTTP 400 (the exemplar key's 115 entries were fetched in the signing-key census above; including them, 543 fetches, 430 matched, 113 unserved); the unserved payloads whose age position is visible sit at the old end of the record—the exemplar key's five earliest entries; the guide's own 2025 worked example has lapsed (§4.5). Two fingerprints printed in the guide (kernel `6052f352...`, initramfs `5f884b69...`) appear in no current reference-values file—the guide's own example entries for those layers embed the current keys—so the published key inventory already shows drift, a point §4.4 returns to. The app policy keys account for 509,185 of the entries, and their payloads settle the policy question of §5: every retrieved policy endorsement is an Oak statement with subject "policy" pointing to a binary-protobuf policy artifact (21 second-level artifacts fetched—11 policy artifacts: 3 Gboard and 2 per other app key; plus 10 KMS-layer statement digests as controls—all hash-matched). Classification method: JSON statements were parsed field by field, and every retrieved payload and all 21 second-level artifacts were string-scanned for `_MAX_ZCDP`, `ZCDP`, and program markers (`def`, `import`, `python`, `program`). A string scan cannot see encoded content, so we then subjected the policy artifacts to a recursive wire-format decode (2026-11-04; decoder and record in the artifact, `scripts/e10_wire_decode.py` and `records/E10-E11-results.md`): a fresh retrieval on the census's sampling (derivation below) served the payloads referencing 57 distinct artifacts; every artifact was fetched, hash-matched, and decoded field by field at every nesting depth, with every long encoded field additionally scanned after base64 and hex decoding and a compression-magic scan throughout. The retrieval behind the decode is a fresh draw, distinct from the census classification sample above: re-running the sampling over the six app policy keys (exhaustive at ≤120 entries, else 30 evenly spaced, plus the guide's named Gboard entry) attempted 159 payload fetches, of which 149 were served and hash-verified (8 returned HTTP 400 and 2 entry fetches failed; per-key counts are in the artifact record), referencing the 57 distinct artifacts just named. The six keys are Gboard policy keys 1 and 2 and the Google App, Pixel Search, Private Compute Services, and Recorder policy keys, and the 159 attempts compose as 31 for Gboard key 1 (30 evenly spaced plus the guide's named entry), 8 for Gboard key 2 (exhaustive at its 8 entries), and 30 each for the other four. **The decode found complete Python workload programs embedded, base64-encoded, in five of the 57 artifacts—three distinct program texts**: a Gboard ACT training program (44,189 B decoded) and a Gboard HMM SentencePiece training program (32,314 B), each carried by two artifacts, and an AGSA/OmniEnt differential-privacy synthesis program (10,676 B) endorsed under the Google App key by current statements—each carrying its `_MAX_ZCDP` guard in code. The remaining 52 artifacts contain no program text in any scanned form—plain, nested, or encoded—and are binary-protobuf configurations; no first-level payload contains a program in any scanned form either. The embedded-program mechanism is therefore verified as documented *and* observed on live policies, the flagship training workloads among them; the October string scan's non-detection is superseded, and we flag the correction. The observation's scope is the retrieved set: exhaustive for the small keys, sampled for the five large ones, and program-carrying artifacts are the minority form (five of the 57 retrieved artifacts; an existence count over the retrieved set, §5). The decode excludes program text in the forms above, not content encrypted under a key we do not hold (no field carries compression or encryption markers). Three denominators in this section should not be conflated: endorsement-statement retrieval attempts (543 across both censuses—430 served, hash-matched, and classified; 113 unserved; exhaustive for keys with at most 120 entries, sampled at 30–31 per larger key); the distinct policy artifacts the decode

sample's statements point to (57, all fetched and hash-matched); and second-level artifacts fetched in the census classification itself (21: 11 policies, 10 KMS-layer controls).

4.2 Endorsement-key inventory

We parsed `access_policy_reference_values/gboard.txtpb` and recomputed each key's SHA-256 fingerprint from the raw DER: **8 unique ECDSA-P256 endorsement keys plus 1 Rekor log-verification key** for Gboard (the guide's 2025 example showed one; that example's key class is unverified, §5). The KMS-layer endorsement keys are published separately, in `kms/reference_values` (stage0, kernel, initramfs, kernel cmdline, system-image, and two container-layer keys). The keys are not anonymous indices: the public key embedded in the exemplar entry of §4.1 decodes to fingerprint `ea0d1f8ffed9512019a2ec968790263d88ea3324c6a6a782114e5ea1be4fd38f`, which is exactly the guide's published *application* endorsement key—so an entry's signing key is identifiably a published endorsement key, and the §4.1 census by that key is an endorsement-key census (this identification is what §4.4's enumeration analysis rests on). Key inventory growth is publicly observable; the key row scores P, because rotation itself is only inferred (the 2025 baseline's key class is unverified, §5). The reference values also confirm the freshness policy: endorsements accepted only if integrated after 2025-02-15 and within 90 days of verification (`signed_timestamp` constraints); this 90-day window is distinct from the endorsement's own 45-day validity above.

4.3 Reproducible-build attempt

The strongest checkable claim is that policy-listed binaries are reproducibly buildable from OSS source. We attempted the build our frozen protocol specified: `bazelisk build -c opt //kms:main` (Bazel 8.2.1, repo-pinned) on a 2-vCPU/7GB-RAM VM. The build failed *before compilation*, at dependency download: Bazel's Java HTTP client throws `NoSuchElementException` in `doTunneling` against our environment's authenticated egress proxy (exit 37; log in artifacts). We record this as **attempted, environment-blocked**—not as evidence against Google's claim. Two further attempts on 2026-11-04 ran the same command on a documented-class cloud environment (4 vCPU/30 GB, unproxied) and failed at target resolution, before any compilation: the first tracked the repository's current HEAD, at which the `kms` package has been deleted upstream (`-deleted_packages`); the second, at the pinned exemplar-era commit (`fe439180...`), failed identically because the root `.bazelignore` at that commit also excludes the `kms` directory—a standalone Bazel module the project's own build compiles from within its own directory—so the plain command does not resolve `//kms:main` at the era tree either. Neither executed the documented build of the exemplar-era code; neither is a result on the claim. The repository's own "Build and attest" workflow—the workflow in this binary's provenance chain—documents the actual invocation: the `kms` directory is a standalone module root, built from inside itself via `bazelisk run -c opt //:install_release_binaries`, which installs the application bundle as `kms/container.tar`. A third cloud attempt on 2026-11-04 ran exactly that invocation at the pinned exemplar-era commit on a hosted cloud environment (Kaggle kernel, 2026-11-04, ≈36 min, 3,813 actions, exit 0; the host publishes no per-kernel resource class for the run—a gap in our own run record that we flag rather than fill): **the build completed (exit 0), and the SHA-256 of the produced `container.tar` equals the endorsed KMS digest `d14a1852...` exactly**. The documented-build claim is therefore verified by outsider execution for the exemplar-era tree—via the project's CI invocation. One precision: the working invocation is documented in the repository's public "Build and attest" workflow file—the workflow the provenance statements name as builder—not in the prose documentation or the audit guide; following the prose docs alone leads to the plain command, which does not resolve at the era tree (second attempt above). We count the result as verification by the project's documented-in-CI procedure and, as §4 states, as an exploratory rescue. By *independent reproduction* we mean a from-scratch build of the same source tree by a party other than the vendor's CI, compared bit-for-bit against the endorsed digest: this execution is one; the 20 provenance statements of §4.1 are the vendor's CI asserting the same digest, not independent reproductions. Three caveats: (i) the execution is a single run at a single commit—the exemplar's—so bit-identical reproducibility is established for that tree, while input-sensitivity across the 20 provenance commits of §4.1 remains observational (identical digests, most from commits that did not touch the screened build inputs); (ii) the bundle form matters: the endorsed digest is over the container bundle that invocation produces, which is the artifact the endorsement names; (iii) the run is third-party-reproducible as a procedure—the invocation, the pinned commit, and the workflow are all public—but the run record does not pin the hosted environment's resource class, so what stands is one independent reproduction, executed once and not yet repeated by a second party.

4.4 Log scale and the enumeration problem

Binary-searching Rekor's index-retrieval endpoint, we measured the public log at ≈**3B** entries on 2026-10-04—an order-of-magnitude figure: both measurements came through that endpoint, and every quantity derived from it in this section (rates, window size, scan durations) inherits that status—growing ≈**1.4B** entries over the 131 days since 2026-05-26 (the May-26 baseline is inferred from the worked-example entry's own index, not an independent measurement): ≈**10.7M entries/day** (≈124/sec, both order-of-magnitude; our 2026-10-08 re-measurement implies a higher short-window rate—bursty traffic in the shared Sigstore log, so the 131-day average is the stable figure). (Methodological note: when we re-ran this measurement on 2026-10-05, the `logIndexes` endpoint that had worked the previous day returned empty results for all queries, including previously valid indices. We did not diagnose the cause and draw no conclusion from it. The two measurements agree on scale; no derived figure is used as a firm input anywhere.) This measurement matters because the paper claims third-party auditors "can also know the full set of potentially active binaries and workloads" from the log [1]. Enumerating that set requires finding entries *by endorsement key*—and here our first pass erred; the correction changes a score. Rekor's key index is keyed by the public key that signed each log entry. Our first pass assumed endorsement keys are not such keys—that they sit only inside entry payloads—and tried the entry-retrieval endpoint with an endorsement-key PEM, observing

timeouts; the search UI (email/hash/commitSHA/UUID/logIndex queries, no key query) reinforced the impression. Both were the wrong instrument. The dedicated index endpoint `POST /api/v1/index/retrieve` accepts a public key and returns its entries: queried on 2026-10-15 with the key embedded in the exemplar entry, it returned 106 entries sharing that key (115 on 2026-11-04; the census of §4.1 uses the later count). And that embedded key is identifiably a *published endorsement key* (§4.2): the guide prints its fingerprint, finds this very entry by searching that fingerprint with `rekor-monitor`, and queries the Rekor BigQuery dataset by `KeyDigest`. Endorsement entries—for binaries and for policies alike—are therefore discoverable by published endorsement key through three documented routes: the index endpoint (no credentials—this is the route our score rests on), `rekor-monitor` fingerprint scans, and the BigQuery public dataset (which requires a GCP account). The all-keys census of §4.1 is the executed form of that discovery: 19 reconstructable published keys → 509,569 entries, payload-verified as described there, via single index calls that return the endpoint's returned list for a key. We therefore score the enumeration claim in three parts. What the *published* keys have endorsed is enumerable, and we executed it: **P** (row C3)—not V, for three stated reasons. (i) Two fingerprints printed in the guide could not be queried at all (no key material, §4.1), so even the published set is not enumerable exactly as published. (ii) Enumeration finds *entries*; reading what they endorse requires the payloads, and payload retention lapses (§4.5): an enumerated historical endorsement may no longer be retrievable. (iii) For the largest policy keys our payload classification is sampled, though entry counts are exact. The relevant inventory is not every key that might exist but the *acceptance set*: the reference values against which client software validates endorsements before trusting an upload. That set is public—it is our census frame—and a key outside it would not be accepted by conforming clients; the bounded residual inside it is a published fingerprint with no reconstructable key material (two, §4.1). The further claim—that the acceptance set as deployed on devices is exactly the published one, i.e. nothing was endorsed under an unpublished key that devices would accept—scores **T** (row C3b): no outsider procedure can reach it, and the inventory is vendor-asserted and already shows drift (two guide fingerprints match no current reference-values key, §4.1; keys published anywhere else would be missed by construction). A third part, separated at this revision: that the index endpoint's *returns* are complete against the log's tree—omits no entry for a queried key—scores **T** (row C3c). The seeded cross-check of §4.1 authenticates returned entries, but a sample drawn from the returns cannot show the endpoint omits nothing. One bounded cross-route reconciliation was executed under a 2026-11-05 amendment: over 1,432 consecutive log indexes, the log-entries route found exactly the index route's 61 entries of one policy key, every index accounted—no omission in that window; outside it, and for the BigQuery and `rekor-monitor` routes, no count check was run. Row C3's P is a retrieval result: enumeration over the returned set, exactly as complete as the endpoint's answer. The split applies our §7 predicate to each part separately. Two claims travel under the word *enumeration*, and our scores separate them: *historical auditability*—what was ever endorsed under the published keys (the census above; bounded by payload retention)—and *current eligibility*—which endorsed workloads could still run, additionally bounded by the 45-day validity and 90-day freshness windows whose enforcement is trust-required (row C4b). Row C3 scores the first; the vendor's "complete set of potentially active workloads" phrasing asserts the second, which inherits the first's limits plus C4b's. What does *not* bound enumeration is raw log scale. The slow paths are the alternatives: `rekor-monitor`'s full scan (the guide itself warns it "can be quite slow") and naive per-index retrieval. At ≈10.7M entries/day the 90-day freshness window holds on the order of a billion entries; on 2026-11-04 we measured batched sequential retrieval through `log/entries/retrieve` (10 indexes per request; 10 batches, 100 entries) at ≈9.7 entries/sec from an unprivileged vantage point (10.3 s per 100), and at ≈5.0–5.7 entries/sec in a three-run replication from a second vantage point (20.0–17.7 s per 100, slowest to fastest): the rate is vantage-dependent, and either rate puts the naive scan of that window at an order-of-magnitude ≈3–6 years (the naive path's cost, not the enumeration route's). Our original estimate used stipulated, unmeasured rates (500–2000 entries/sec); it is superseded by this measurement and should not be cited. The distinction that survives is narrower than our first pass claimed: the log lets anyone enumerate what the *published* keys have endorsed—demonstrated—while no log can show that nothing was ever endorsed under an *unpublished* key; that is the custody question of §5, scored there. The route the census rode on is perishable: it is Rekor's v1 index endpoint, which the v2 design removes (Appendix C)—no v2 equivalent of the endpoint exists, so were the v1 index retired, the census would be reproducible only from the archived raw outputs in the artifact package, the durable record; the census stands as executed, but its live reproducibility lasts only while v1 is served.

4.5 Retention and freshness

Three time limits govern checkability: the endorsement's validity (45 days for the worked exemplar), the 90-day freshness window in the reference values, and the transparency store's retention (at least one year). Table 4 (Appendix B) collects them.

The guide's February-2025 Gboard policy worked example (endorsement `d72534e0...`, policy `a324fe96...`) no longer resolves from the transparency store (checked 2026-10-05)—consistent with the documented "at least 1 year" retention. The lapse is not confined to that example (both payloads still returned HTTP 400 on 2026-11-04): across the KMS-layer keys of the §4.1 census a substantial share of older payloads is unserved—one published key (Gboard policy key 2) has all eight of its payloads unserved, none retrievable (its entries and key remain public)—(24 of 51 stage0, 15 of 43 kernel, 29 of 83 system-image payloads), while recent payloads serve normally—retention expiry is advanced but uneven across keys and layers (Table 3), a cross-sectional pattern rather than a single cliff (we did not stratify availability by entry age across keys; for the exemplar key the five unserved payloads are its five earliest entries, so the timescale rests on the documented floor, not a measured curve), and it is already consuming the historical record.

5 Findings

Across the condensed matrix (Table 1, Appendix B), 2 of the 24 vendor-assertion rows score V, as do both disclosure rows—4 of 27 rows in all; 10 score P—six executed with qualified answers, two

inspection-only (C1, C2), one partially executed (E1), and one, C5, a negative finding (the executed check contradicted the claim as worded, §3), not a partial success—and 13 score T—so the headline of this section is narrow: the central *chain* checks out. No V attests runtime behaviour: two are executed artifact verifications (the transparency-log chain; source and build recipe); two are verified statements whose claims are the statements themselves. The full matrix (Appendix B) has 29 rows scoring 4 V / 10 P / 15 T; the two tables reconcile exactly: from the full 4 V / 10 P / 15 T, two merges (A4 into the A3 row; F2 into the B5 row) remove two T rows — arriving at 4 V / 10 P / 13 T over 27 rows. Every other row carries the same score in both tables, and the same claim wording up to the condensed table's abbreviations; B2 and B3 score P in both, G2 T in both. The distribution is sensitive to row construction; we report the variants rather than a single fragile count (the cells are tabulated in Table 10, Appendix B). Merge arithmetic uses one rule: a merged row scores P when its parts disagree and at least one part is checkable (V or P), and the common score when they agree—so the C3 family (P+T+T) merges to P and the C4 pair (V+T) merges to P. Variants are computed on the atomic rows in a fixed order: re-scorings first (merges then use the re-scored values; A4's flip acts through its merge into A3's row), then merges under the rule above, then removals; a re-scoring and a removal of the same row are alternatives, never stacked. On that rule: merging the C3 family back into one row would read 4/10/11 over 25 rows; merging the C4 pair, 3/11/12 over 26 rows; scoring the per-round row as the disclosure it rests on, 5/10/12 over 27 rows (not adopted: G2's claim as worded asserts the visibility fact, not its disclosure, so §3's disclosure rule does not reach it—unlike F1 and C4, whose claims as worded are disclosures); scoring the two inspection-only P rows (C1, C2) at the stricter B4 line (T; neither executed on a live policy byte)—4/8/15; and merging the split pairs under a weakest-link rule (weakest part's score, not the checkable component): C3 family 4/9/12 over 25 rows, C4 pair 3/10/13 over 26 rows, both together 3/9/12 over 24 rows. Removing G4 alone—the one row that scores no vendor claim, retained as a scope limit on B2—reads 4/10/12 over 26 condensed rows; removing G4 together with F2—a vendor claim scored T as not testable today, present as a row of its own only in the full matrix—reads 4/10/13 over 27 full rows. Those variants vary construction alone; we therefore cross them with the other two axes the paper documents. Scorer variance: the blind automated re-scoring (§3) agreed with every score it covered—all 27 rows of the as-executed matrix—but recorded four rows where a competent scorer could land elsewhere—each an up-flip the adjudication considered and rejected: A3 and A4 at P, not T (row A2's inspection precedent); B2 at V, not P (its executed instance was bit-identical); G4 flagged P, not T (a build-comparison procedure exists but was not executed; not adopted—G4 scores no vendor claim, so there is no claim for a procedure to decide, Table 1). The published scores are the adjudicated ones, and the scorer axis crossed here is the strict direction only: none of the four carries a V and three already sit at T, so the one down-flip that moves any letter count is B2 (P→T), which crossed with the strict C1/C2 line reads 4/7/16. The rejected up-flips lie outside that axis, and adopting one moves the counts the other way: A3 at P (A4 likewise, through its merge into A3's row) crossed with the C4 merge reads 3/12/11 over 26 rows—a P count of 12, above the envelope reported below—and B2 at V would read 5/9/13 on the published construction; G4 at P on the same construction would read 4/11/12 over 27 rows. Amendment state: the snapshot counts below (3/10/12, defined below) crossed with the G2-disclosure variant read 4/10/11 over 25 rows, and with the strict line 3/8/14 over 25 rows. The construction and scoring variants above act on disjoint rows of the published matrix, so they stack; the envelope we report is their joint envelope over all stacks—not only the crosses named—of the published matrix at its present, post-amendment scoring state: exactly its domain. Stacking the three strict readings—the strict C1/C2 line, the B2 flip, and the weakest-link C3 merge—reads 4/6/15 over 25 rows, and the weakest-link C4 merge stacked on top reads 3/6/15 over 24 rows. Across that joint envelope—the scorer axis crossed in its strict direction only—the verified count is 3–5 and the partially checkable count 6–11 (ranges coordinatewise, not jointly attained): the fragility sits in the P/T split, not in the verified count. The amendment-state cells are not envelope members—they record superseded scoring states—and crossing them with the construction variants reaches verified counts outside the envelope (2/11/11 over 24 rows, the snapshot with the C4 merge; 2/10/12 over 24, the snapshot under the weakest-link C4 merge; 2/12/12 over 26, A1 reverted alone with the C4 merge; among others): every sub-3 cell pairs a presentational variant with a reverted score, so no reading of the published matrix at its present state goes below 3. Two of the scores are amendment-raised (A1, C3): direction-aware follow-ups, frozen before their runs but written after the October results were known (§4); they should be weighted as replications awaiting blinded repetition, not as confirmatory results. One transform of the published matrix is worth printing—the snapshot: revert the two amendment-raised scores (A1 to P: edges hash-verified, signatures not yet authenticated; the enumeration claim to its single pre-split row at T; 25 rows, not this section's 27) and the counts read 3 V / 10 P / 12 T. B2's letter is unchanged under it. Nor is it the exercise's own constitution (Table 9: the C3 family, the C4 pair, and the A2 pair each one merged row; 2/13/8 over 23 rows as scored, 2/11/10 with its two adopted disagreements applied). The rows are also not homogeneous in type, and the headline counts should be read with their types: of the 27 condensed rows, 24 score vendor assertions about the system (2 V / 10 P / 12 T); two score disclosures whose content is the statement itself (the freshness window as published, C4; the side-channel limitation as acknowledged, F1—both V); and one (G4) scores no vendor claim at all—it records the vendor's own reproducibility exclusion, retained as a scope limit on B2 (T).

What checks out (V). The log→endorsement→binary→provenance source chain is real and public for the documented exemplar (as of 2026-10-04/08), at two different strengths, stated separately: the log→endorsement→binary edges are hash- and signature-verified end to end, and the rebuild edge was executed on 2026-11-04 (§4.3); the provenance edge is digest equality over provenance statements whose own signatures we did not independently verify, which is why row B3 scores P, not V. The full chain is exemplar-scoped: for independently sampled entries, the verified link is entry→payload (§4.1); inclusion, provenance, and rebuild were executed for the exemplar only. Log membership is verified: the inclusion proof folds to the signed checkpoint (§4.1), reproduced by a second implementation on 2026-11-04. The build recipe is public and fetched (V). Endorsement keys are public and their fingerprints recompute, but

the key row scores P: the Gboard policy-key set grew from one key (the 2025 guide's example) to eight—growth that does not by itself establish rotation (the 2025 example's key class is unverified). The 90-day freshness window splits: the window as published is verified (row C4, V); its enforcement inside TEEs is unobservable (row C4b, T). The 20 provenance statements are consistent with toolchain determinism, but equally with build caching; input-sensitivity is not established. Side-channel limitations are disclosed in the paper itself, and that disclosure is itself the verified claim (row F1). The "best-effort" TTL (time-to-live; row D3) and the GCP Confidential Space offload path's carve-out from the reproducibility claim ("not guaranteed to be reproducible", row G4) are likewise disclosed in the paper itself, but both score T, not verified disclosure: each states a limit on the vendor's own behaviour that the reader must take on trust.

What is degraded (P)—and two rescorings. (1) Enumeration splits into three claims. What the *published* keys have endorsed is enumerable, and we executed it (509,569 entries across 19 reconstructable keys, §4.4): it scores **P**, not V, because two guide fingerprints are unqueryable for lack of key material, payload availability decays under retention, and our payload classification for the largest keys is sampled. "Enumerable" throughout this paper means the index endpoint returns the entry set for each published key and fetched payloads hash-match their entries; completeness is judged on that returned set, not on whether every payload is still served—payload lapse is the retention finding (§4.5), not a gap in the enumeration. The endpoint itself was cross-checked against the log's signed tree on a seeded sample (24 entries, six per key, drawn by a seeded uniform sample from the live index returns for four keys): every sampled entry's inclusion proof folded to the checkpoint root served with it, and every checkpoint signature and signed entry timestamp verified—so the returns are authenticated log entries, though a sample drawn from the returns cannot show the endpoint omits nothing; that the returns are complete against the tree is a third, distinct dependency (row C3c) and scores **T** (§4.4). Whether the published inventory is *complete*—that nothing was endorsed under an unpublished key—is a separate claim (C3b) and scores **T**: no outsider procedure can reach it. (2) Live access-policy *content*: policy endorsements are retrievable in bulk through the policy keys, and we retrieved and hash-verified them (§4.1)—and the embedded-program mechanism the system paper describes for Python workloads is exercised in production: recursive wire-level decoding of the 57 distinct artifacts a fresh identically-sampled retrieval references found complete Python workload programs embedded (base64-encoded) in five of them—the Gboard ACT and HMM SentencePiece training programs among them, each with its `_MAX_ZCDP` guard in code (§4.1). Row B4 stays P on the sampling bound—the large keys are sampled, and only served payloads could be decoded (149 of 159 attempts)—not on the 5-of-57 share: that denominator mixes workload types (the text does not isolate how many of them are of a type that could carry a program), so it cannot carry the score; the remaining 52 artifacts are binary-protobuf configurations, with no program text in any scanned form. And "inspectable" in B4's wording means inspectable by an external analyst applying recursive wire-format decoding, as here: neither the policy bytes at surface level nor the vendor's documented string-level inspection path reveals the programs, and whether the deployed serving path itself decodes them is not something we test. The embedded programs carry their own `_MAX_ZCDP` guard in code; that a guard exists shows the program text self-limits a budget parameter—whether the guard's value matches any budget the surrounding policy states is a separate claim, and we do not make it. Our October string scan reported non-detection; the decode supersedes it—a string scan cannot see encoded content, and we flag the correction. (3) The reproducible build's place in this list changed kind, not letter: its executed core is now verified (the project's CI invocation, run on a hosted cloud environment, reproduced the endorsed digest exactly, §4.3)—via an exploratory rescue of the frozen protocol, not a frozen-protocol confirmation—and row B2 remains P only because the claim as worded covers all endorsed binaries and one was executed; the cross-commit input-sensitivity question rides inside the same row's caveats (§4.1).

What cannot be verified without Google (T)—the residual. This enumeration is the audit's core deliverable:

- *Endorsement-key custody.* The log shows key *usage*, never custody. The residual trust in this audit is not a single anchor but a typed set of dependencies—endorsement-key custody (Google), hardware attestation roots (AMD/Intel, row E2), log operation (Sigstore/OpenSSF, under the collusion assumption of §2), and authenticated source distribution (GitHub)—and they fail differently. For the endorsement key specifically: a compromised Gboard policy key can authorize arbitrary *future* policies, defeating the verifiability chain for data bound under them (and, with operator collusion, the remaining runtime checks as well); it cannot rewrite retained log history, forge the KMS binary's provenance retroactively, or defeat devices' reference-value checks on the KMS itself. Detection depends on monitors watching the log by endorsement key—which works (`rekor-monitor` or the index endpoint, §4.4)—but only for published keys, and only while payloads are retained.
- *Runtime behavior.* That DP noise is actually added as programmed, that the KMS releases keys only to matching TEEs, that results release atomically with pipeline-state updates, that the Noise root↔worker channel holds—all code-inspectable, none externally observable by the no-device persona of §3. Parts are *conditionally* verifiable: a participating device can validate the KMS's attestation against the reference values under row E2's hardware assumptions—outside our persona. Our T therefore reads "no procedure for this persona," not "unverifiable in principle."
- *Sideloaded proprietary logic.* Model architectures and preprocessing protos loaded at runtime are deliberately not in the log. The paper's assurance is that auditors can inspect *how these are used*, rather than the proprietary values themselves. That inspection is not currently possible from public artifacts: the interface contracts that would show which sideloaded fields can influence data flow are not published, and the artifacts are not in the log, so for our persona the distinction between privacy-relevant and merely proprietary sideloading is not decidable. Two things would decide it: published interface contracts (recommendation (3) in §7), or a confinement argument that arbitrary sideloaded

transforms remain inside the policy's clipping, accounting, and output bounds. Neither is available, so the characterization that all privacy-relevant logic is hardcoded stands as trusted.

- *Upload-key binding and TTL (row D3).* Two propositions share this row. That uploads are bound to their policy's key is a cryptographic binding—code-inspectable, with its enforcement inside the KMS TEEs and so unobservable. That keys expire after their TTL rests, by the KMS design's own account, on replicated observations of an untrusted time source plus key erasure, and the design states the limitation explicitly: elapsed-time assurance, not the binding, is the trust-required part.
- *Hardware roots and side channels.* AMD/Intel attestation roots are trusted, not verified; side-channel observability is acknowledged in the paper itself [1] (see also SNPeek [12]). Bell-Clark et al. [13] identify insider-exploitable side channels in this exact platform that can circumvent DP guarantees, which sharpens the Adv. 2 residual above: the side-channel surface is not hypothetical. (The analysis is Google-first-party; it also reports DP as mitigating two of the channels—one implemented in OSS—so the risk applies to the remainder.)
- *Operator views.* "Only metrics and DP weights are visible to the operator" is not checkable without device instrumentation and operator-side audit—though the paper honestly discloses that the operator *does* see which uploads are used per round (§6.3 of Daly et al.), forgoing sampling amplification.

From checks to the privacy claim. The matrix scores checkability claim by claim; the advertised guarantee, however, is a composition. Table 7 (Appendix B) maps the obligations the DP guarantee is made of onto what this audit establishes about each. The first two columns are phrased generically, but their transfer to other pipelines is untested; the entries are this deployment's, as of the audit dates.

Against our threat rubric: Adv. 1 (honest-but-curious operator) is substantially constrained on the audited chain by the crypto binding as designed and code-inspected—the binding's in-TEE enforcement is scored at row A3, not here; Adv. 2 (malicious operator) is constrained but not eliminated (runtime unobservable, side channels open); Adv. 3 (compromised endorsement key) defeats the verifiability chain for data whose release the key authorizes—it cannot rewrite retained log history or forge the KMS binary's provenance retroactively—and, with operator collusion, the remaining runtime checks as well; it is detectable via log monitoring by published endorsement key. The consequence rule we apply throughout: endorsements whose log inclusion precedes a compromise stand on their own proofs—inclusion, checkpoint, and endorsement signatures verify without trusting the key's later state—while endorsements first appearing after it must be treated as unendorsed; on a suspected compromise, a verifier bounds the window from the log's timestamps and re-checks every chain relying on entries inside it; the window opens where suspicion starts, not at the compromise itself—onset is unobservable, so uncertain-interval entries are re-checked, not presumed clean. Policy digest-pinning constrains Adv. 5 (malicious workload author) *except* through sideloading, whose contents are not in the log at all (§5); Adv. 4/6 are carried as trust-required residuals, with no constraining mechanism claimed. Table 5 (Appendix B) collects this mapping.

6 Comparative Case: Apple Private Cloud Compute

Apple PCC is the closest production confidential-computing deployment. We juxtapose it against Google's confidential federated compute (CFC) on the same dimensions, from Apple's security documentation and an independent WiSec'26 analysis [10]. One confound runs through it: PCC is inference-only and CFC a training system, so several dimensions—policy language, workload—differ by workload type as much as by verifiability design. PCC publishes every production build to an Apple-operated transparency log, ships a Virtual Research Environment for log-consistency checks, and devices refuse unlogged builds. *But Apple explicitly does not provide reproducible builds* [10, 11] and ships partial source only; Apple makes no claim that the published source corresponds to the released binary. Google's deployment is broader in claimed scope (production *training* with DP, full OSS, reproducibility machinery), but its broadest checkability claim—enumerating the complete workload set—rests on the vendor's key inventory and on payload retention, while Apple's claim is narrower—attestation-to-log binding for an inference-only service.

7 Discussion and Recommendations

For operators claiming "externally verifiable." (1) Keep the published endorsement-key inventory complete and current, in the channel clients actually consume, so the acceptance set and the audited set cannot drift apart unnoticed. Two fingerprints in the audit guide already match no current key. (2) Extend transparency-store retention for *policies* beyond one year, or archive them somewhere content-addressed and public; otherwise the "complete set of workloads" claim decays on the documented "at least 1 year" retention floor. In falsifiable form: an archive is a remedy only if every policy endorsement resolves to an archived payload—the omission check our census already runs (§4.1), currently failing on retention (113 of 543 attempted payloads unserved), and only within the enumeration frame of the published keys (rows C3b/C3c). (3) Publish the sideloading *interface contracts* (which config fields are privacy-relevant) so auditors can check the "hardcoded" claim without seeing proprietary values; the contract supports a concrete check only if it enumerates every field that can influence data flow; the contract enables the check—compliance still takes the confinement argument of §5. (4) Document the build environment's resource class—the CI workflow's runner class is documented as 4CPU/16GB/150GB, while our rebuild's hosted host publishes none (§4.3)—*and* provide a minimal rebuild target.

For the research community. Treat "externally verifiable" as a testable predicate, we propose. *A claim is externally verifiable iff an outsider can execute a documented procedure on public artifacts and reach a yes/no answer.* A *no* is an answer too: where the executed procedure contradicts the claim as worded, the row records P* (row C5)—a decisive negative finding, not a partial pass. By that test, most claims in our extracted corpus do *not* fully pass: of the 27 condensed rows, 4 meet the predicate outright—2 among the 24

vendor-assertion rows, the other 2 verified disclosures (§5). The 10 P rows divide as in §5 (six executed-qualified; C1/C2 inspection-only; C5 a negative finding; E1 partially executed); the remaining 13 have no outsider-executable procedure.

Limitations. Scope first: the audit covers a single deployment. The primary window is 2026-10-04/08, with re-probes on 2026-10-15, the 2026-11-04 amendment experiments of §4, and a 2026-11-05 endpoint re-probe (Appendix C). Counts differing between October and November (e.g. 106 vs 115 signing-key entries) reflect log growth. We executed no device-side run: row A2's evidence is the executed static inspection of the public client code path (§3); that production devices run that verifier is untested. The from-scratch rebuild ran once—a hosted cloud environment (Kaggle), the exemplar-era commit. Statements about the other 19 provenance commits rest on provenance observation, not on rebuilds of those trees. The Apple comparison is document-based, not a worked audit. The claim corpus is itself a single-auditor extraction from the fixed sources of §3: no second, independent extraction was performed, so its recall is unmeasured—a checkable claim the extraction missed would appear neither as a row nor as a T score. The risk is visible inside the audit: the composition map (Table 7) surfaced one obligation the guarantee composes from—DP output gating with atomic release—that has no checkable claim or artifact, hence no row. Every count here is over the extracted corpus, and the predicate test above is a statement about that corpus, not about the vendor's claim set as a whole.

8 Related Work

TEE-based auditability has been studied for *inference* and *benchmarks*, not production training. The system we audit builds directly on Eichner et al.'s confidential federated computations design [6], and Cheu et al. [7] extend the same platform toward provably private analytics. Schnabl et al. [8] ("Attestable Audits," TAIG@ICML'25) run safety benchmarks inside TEEs with results published to a transparency log—closest in spirit, but a prototype for evaluation, not a training pipeline. Siby et al. [9] (GuaranTEE, EuroMLSys'24) target attestable ML via Arm CCA. Oak's Transparent Release [5] provides the endorsement machinery Google builds on; Sigstore/Rekor [4] the log. On the hardware side, flexHEG [16] sketches hardware-enabled guarantees for AI compute—largely roadmap, not deployment. Most on point for our dimension 6, Bell-Clark et al. [13]—Google-first-party authors—identify side channels in Google's own confidential federated compute platform an insider could exploit to circumvent DP—directly relevant to the Adv. 2 residual we score. Industry deployments (Apple PCC [10], Google Private AI Compute [14], Meta Private Processing [15]) publish varying transparency artifacts; to our knowledge none has received an independent outsider audit of its *training*-pipeline verifiability claims. The survey behind the priority claim: web and arXiv sweeps on 2026-10-06/14/17/20—a scoping search, not a systematic review—for outsider audits of this pipeline or comparable verifiability claims; the nearest misses were vendor-engaged assessments of an adjacent inference system (Private AI Compute) and zero-knowledge protocol proposals—neither an outsider audit of a production training pipeline.

Eskandarian et al. [19] systematized Certificate Transparency's monitor/auditor roles; Meiklejohn et al. [20] showed that deployed CT's inclusion promises are rarely verified by the clients who rely on them. Where that literature audits certificate logs, we audit an endorsement log binding binaries and policies for a training pipeline; our all-keys census is, in their terms, a monitor's enumeration from the published key inventory. LogPicker [21] strengthens CT logs against log-operator misbehaviour by cross-witnessing—the design answer to the collusion residual we can only enumerate; Rekor has no equivalent witness protocol, so the residual stands as scored.

A parallel line pursues the same goal through cryptographic proof: verifiable aggregation (e.g. Xu et al.'s VerifyNet [17]) and proof-of-training schemes (e.g. Jia et al.'s Proof-of-Learning [18]). This line is out of scope: no proof-based system is, to our knowledge, deployed in production with comparable verifiability claims; reuse of our checklist is untested.

9 Conclusion

We built a seven-dimension audit checklist for sealed-hardware training pipelines, and applied it—as a single auditor, from public artifacts alone—to Google's production TEE-based FL system. For the documented exemplar, the central chain checks out end to end—its log and endorsement edges recomputed by us, its provenance edge checked at digest equality, its rebuild edge on the single exploratory-rescue execution of §4.3; beyond the exemplar: every served payload we fetched hash-matched its entry (430 of 543 fetches, zero mismatches; the 113 unserved payloads are the retention finding), with proof-level authentication on the cross-check sample (§5). An outsider *can* enumerate what the published endorsement keys have endorsed—for every reconstructable published key, as returned by the v1 index endpoint (row C3c)—but the enumeration is only as complete as the vendor's published key inventory (itself trust-required, row C3b); its score (C3), like A1's, is amendment-raised—a replication awaiting blinded repetition, not a confirmatory result; and history expires: endorsed payloads lapse from the store (floor "at least 1 year"; the guide's February-2025 example was unserved by October 2026), so today's verifiable record decays unless archived. Beyond those failures lies the defined residual—endorsement-key custody, runtime behavior, sideloaded logic, hardware roots—none reachable by this audit's public-observer procedures: custody and runtime behavior would need Google's cooperation or a participating device, sideloaded logic its unpublished contracts, hardware roots trust in AMD/Intel. We publish the framework, the procedures, the hashes, and the residual list so the next audit is faster.

Competing Interests

The author declares no competing interests. No funding was received for this work.

Acknowledgment

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

AI Use

The author used the AI tool Muse Spark (version 1.3, Meta) for drafting and editing prose and for writing audit scripts. The contents are reviewed, edited, and attested for accuracy by the author.

Open Science

The full audit code, measurement logs, and reproduction scripts are archived in the anonymized artifact at https://anonymous.4open.science/r/sealed-hardware-fl-audit-835E/ (available until October 2027), hash-pinned by the artifact's SHA-256 manifest (`SHA256SUMS`, at the artifact's root); the author maintains the anonymized copy in the interim, and a permanent copy will be deposited on acceptance. The wire-decode and client-inspection records are `records/E10-E11-results.md` in the artifact; the index cross-check is `records/E13-results.md`.

Ethical Considerations

This work audits *public claims using public artifacts*. We accessed only endpoints intended for public use (transparency logs, public code hosting, public APIs), stayed within rate limits, and did not attempt to access non-public data, probe private infrastructure, contact Google, or interact with any user devices or data. No external ethics review was sought: the work involves no human subjects, no personal data, and no interaction with the operator's systems beyond public endpoints, so we judged that no protocol required external approval. Our findings classify as follows: two auditor errors, corrected in §4 (the index-space reading and the key-index premise); availability observations (payload retention lapses, §4.5); documentation drift (two guide fingerprints match no current key, §4.1); and assurance limitations (the §5 residual). None is a vulnerability in the audited system: no finding shows a mechanism failing against its stated specification, and the two that initially appeared to were our own misreadings. Had any finding met that bar, we would have followed coordinated disclosure before publication. The audit is non-adversarial toward individuals: the entities it audits are public organizations, and its objects published artifacts. A risk we considered: publishing exact procedures for enumerating endorsements could aid evasion of monitoring—we judge this minimal, since the procedures mirror Google's own published audit guide and disclose nothing an insider could not already do. A second risk is overinterpretation: the census shows what the published keys have endorsed, not that no other endorsements exist (row C3b, trust-required). The document-based juxtaposition of Apple PCC relies on public documentation and a peer-reviewed independent analysis; no Apple systems were probed.

References

[1] Katharine Daly, Yu Xiao, Zachary Garrett, Brett McLarnon, Jianpeng Hou, Arun Ganesh, Yanxiang Zhang, Noriyuki Takahashi, Haicheng Sun, Yuanbo Zhang, Timon Van Overveldt, and Daniel Ramage. Toward provably private learning from federated data. *arXiv:2609.31494 [cs.CR]*, v4, September 30, 2026. https://arxiv.org/abs/2609.31494.

[2] Katharine Daly and Daniel Ramage. Toward provably private learning from federated data. *Google Research Blog*, October 2, 2026. https://research.google/blog/toward-provably-private-learning-from-federated-data/.

[3] Google Parfait. confidential-federated-compute. *GitHub repository*, 2026. https://github.com/google-parfait/confidential-federated-compute. Docs: `docs/inspecting_endorsements.md`, `docs/inspecting_attestation_records.md`.

[4] Sigstore. Rekor transparency log. https://docs.sigstore.dev/logging/overview/.

[5] Project Oak. Transparent release / endorsement documentation. https://github.com/project-oak/transparent-release.

[6] Hubert Eichner et al. Confidential federated computations. *arXiv:2404.10764*, 2024.

[7] Albert Cheu et al. Toward provably private analytics and insights into GenAI use. *arXiv:2510.21684*, 2025.

[8] Christoph Schnabl, Daniel Hugenroth, Bill Marino, and Alastair R. Beresford. Attestable audits: Verifiable AI safety benchmarks using trusted execution environments. In *Workshop on Technical AI Governance (TAIG) at ICML*, 2025. https://arxiv.org/abs/2506.23706.

[9] Sandra Siby, Sina Abdollahi, Mohammad Maheri, Marios Kogias, and Hamed Haddadi. GuaranTEE: Towards attestable and private ML with CCA. In *Proc. 4th Workshop on Machine Learning and Systems (EuroMLSys)*, 2024. https://arxiv.org/abs/2404.00190.

[10] Apple. Private Cloud Compute security guide, 2024. https://security.apple.com/documentation/private-cloud-compute.

[11] Yannik Dittmar, Marvin Jerome Stephan, Thomas Völkl, Matthias Hollick, and Jiska Classen. Unlocking Apple's Private Cloud Compute: An Analysis of Privacy-Preserving Artificial Intelligence. In *Proc. ACM WiSec*, 2026. https://arxiv.org/abs/2605.24239.

[12] Ruiyi Zhang et al. SNPeek: Side-channel analysis for privacy applications on confidential VMs. In *NDSS*, 2026.

[13] James Bell-Clark, Albert Cheu, Adria Gascon, and Jonathan Katz. Hardening confidential federated compute against side-channel attacks. *arXiv:2603.21469 [cs.CR]*, 2026. https://arxiv.org/abs/2603.21469.

[14] Google. Private AI Compute technical brief, November 2025. https://services.google.com/fh/files/misc/private_ai_compute_technical_brief.pdf.

[15] Meta. Private Processing for WhatsApp overview, June 2025. https://ai.meta.com/static-resource/private-processing-technical-whitepaper.

[16] James Petrie, Onni Aarne, Nora Ammann, and David Dalrymple. flexHEG: Flexible hardware-enabled guarantees for AI compute. *arXiv:2506.15093*, 2025.

[17] Guowen Xu, Hongwei Li, Sen Liu, Kan Yang, and Xiaodong Lin. VerifyNet: Secure and verifiable federated learning. *IEEE Trans. Inf. Forensics Security*, 15:911–926, 2020.

[18] Hengrui Jia, Mohammad Yaghini, Christopher A. Choquette-Choo, Natalie Dullerud, Anvith Thudi, Varun Chandrasekaran, and Nicolas Papernot. Proof-of-learning: Definitions and practice. In *Proc. IEEE Symp. Security and Privacy (S&P)*, 2021. https://arxiv.org/abs/2103.05633.

[19] Saba Eskandarian, Eran Messeri, Joseph Bonneau, and Dan Boneh. Certificate Transparency with Privacy. *Proceedings on Privacy Enhancing Technologies*, 2017(4):232–247. https://doi.org/10.1515/popets-2017-0036.

[20] Sarah Meiklejohn, Joe DeBlasio, Devon O'Brien, Chris Thompson, Kevin Yeo, and Emily Stark. SoK: SCT Auditing in Certificate Transparency. *Proceedings on Privacy Enhancing Technologies*, 2022(3):336–353. https://doi.org/10.56553/popets-2022-0075.

[21] Alexandra Dirksen, David Klein, Robert Michael, Tilman Stehr, Konrad Rieck, and Martin Johns. LogPicker: Strengthening Certificate Transparency Against Covert Adversaries. *Proceedings on Privacy Enhancing Technologies*, 2021(4):184–202. https://doi.org/10.2478/popets-2021-0066.

[22] Open Source Security Foundation. Running Sigstore as a managed service: a tour of Sigstore's public good instance. *OpenSSF Blog*, Oct. 2023. Notes Sigstore began as a collaboration between Red Hat and the Google Open Source Security Team. http://openssf.org/blog/2023/10/03/running-sigstore-as-a-managed-service-a-tour-of-sigstores-public-good-instance/.

A Artifact Package Contents

All audit procedures ran against public endpoints on 2026-10-04/08, with re-probes on 2026-10-15 and amendment experiments on 2026-11-04; no credentials, devices, or private data were used. Independent re-execution is time-limited: the census rests on Rekor's v1 index endpoint, which the v2 design removes (Appendix C), and payload availability decays under the store's retention (§4.5). Artifacts: (1) Python scripts for the Rekor/endorsement/provenance chain (`scripts/`); (2) raw fetched artifacts (Rekor entries, endorsement statements, SLSA bundles) with SHA-256 recomputation logs (`data/`); (3) the Bazel build-attempt log (`data/build_kms_attempt.log`); (4) the audit matrix as executed (27 rows; `AUDIT_MATRIX.md`)—the published full matrix (29 rows) adds two revision-stage split rows (A2b, C3c) to it, recorded in §3 and Appendix B; the published condensed matrix is a different 27-row set again (the full matrix with A4 and F2 merged

away), so the two 27-row matrices differ in composition: the as-executed set contains A4 and F2 as separate rows and predates the splits, the condensed set contains A2b and C3c and not A4/F2; (5) pilot report (`PILOT_REPORT.md`); (6) the 2026-11-04 census, signature-verification, documented-build, wire-format-decode, client-inspection, and index cross-check records, the frozen amendment, both scoring-reliability records (the second-scorer exercise; the automated re-scoring of the as-executed matrix), and the 2026-11-05 cross-route reconciliation record (`records/`, `AMENDMENT-2026-11-04.md`, `AMENDMENT-2026-11-05.md`). The review artifact's contents are pinned by a SHA-256 manifest (`SHA256SUMS` in the artifact, covering every file except the bundled manuscript copy, which itself states this hash); the manifest's SHA-256 is `1614299a34ac1183357af6972994b62760ccbf3e3ffb51b353edd878238dc5b9`, so its contents are pinnable outside the artifact itself. For anonymous review these are provided via an anonymized repository link (https://anonymous.4open.science/r/sealed-hardware-fl-audit-835E/); on acceptance they will be published permanently. Live endpoints queried: `rekor.sigstore.dev`, `search.sigstore.dev`, `federatedcompute-pa.googleapis.com`, `api.github.com`. Rate limits were respected (sequential requests, no parallelism beyond 1).

AI-use disclosure. See the AI Use section (the mandated disclosure under the PoPETs 2027 template); all quantitative claims were recomputed from raw tool outputs by the author, who takes full responsibility for accuracy, originality, and integrity per the PoPETs AI-use policy.

B Full Audit Matrix

The complete 29-row checkable-vs-trust-required matrix is printed in Table 2 (claim names abbreviated; per-claim evidence pointers, procedures, and threat-model scoring are in `AUDIT_MATRIX.md` in the artifact package) and summarized in Table 1; both tables are in this appendix. Table 8 prints the full claim wordings as scored and Table 9 the second-scorer exercise's agreement table—both previously available only in the artifact.

C Reproduction Commands

Note: endpoints as documented in October 2026; re-probed 2026-11-05. Rekor's v1 `/log/info` *still returns 404 (as of 2026-10-15), but the v1 endpoints used below remain live:* `POST /api/v1/index/retrieve` *with the exemplar key returned its 115 entries, and entry retrieval by UUID returned the entry—so the census of §4.4 is still reproducible live as of the re-probe date. The migration risk has since resolved into a fact: Rekor v2 removes the search index by design (it exposes an entry-submission endpoint and tile-based reads only; a separate search service has been announced but is not live), so no v2 equivalent of the* `index/retrieve` *endpoint used in §4.4 exists—were the v1 index retired, the census would be reproducible only from the archived raw outputs in the artifact package. The witnessing and migration discussion in §4.4 likewise concerns the audited v1 deployment; no v2 equivalent is claimed for it.*

```
# 1. Rekor entry -> endorsed hash
curl -sL \
"https://rekor.sigstore.dev/api/v1/log/entries/\
108e9186e8c5677a4e8e68884126228c18\
dbe566ab43eb0be79b077401149bd340bc29\
5d78e9cd86" \
 | jq -r '.[] | .body' | base64 -d \
 | jq '.spec.data.hash.value'
# 2. Endorsement payload + hash check
H=62e818ac02a0427b9811d08468e286a\
ca2f5c68ca44191a9ab1f86075afe5a87
curl -sL \
"https://federatedcompute-pa.googleapis.com/\
data/transparency/sha2-256:$H" \
 -o end.json && sha256sum end.json   # must equal $H
# 3. SLSA provenance bundles for the endorsed
#    KMS artifact digest (GitHub Attestation API)
curl -sL \
"https://api.github.com/repos/google-parfait/\
confidential-federated-compute/attestations/\
sha256:d14a1852f08b528f9975245f300d44e2500a\
aff9c8a9c5572e0114b7203e3e47"
# 4. CI build (workflow-documented runner class
#    >=4CPU/16GB/150GB, unproxied net); executed
#    2026-11-04 on Kaggle (~36 min, 3813 actions,
#    exit 0; the executed host's own resource
#    class is unpublished, S4.3),
#    container.tar sha256 == d14a1852... above
git clone https://github.com/google-parfait/\
confidential-federated-compute
cd confidential-federated-compute
git checkout \
 fe439180bb24e65ad46e869eddee88f9c9b34875
cd kms
bazelisk run -c opt \
 //:install_release_binaries \
 -- --destdir=<output dir>
sha256sum <output dir>/kms/container.tar
# 5. All-keys census (per key; counts in Table 3)
curl -s -X POST \
 https://rekor.sigstore.dev/api/v1/index/retrieve \
 -H 'Content-Type: application/json' \
 -d '{"publicKey": "<PEM of endorsement key>"}'
#    (the endorsement keys are the public keys
#    distributed as the reference-values files of
#    the public cfc repository; their SHA-256
#    fingerprints name the rows of Table 3)
# payload for each returned UUID: GET
# /api/v1/log/entries/<UUID>, then fetch
# spec.data.hash.value from the transparency
# store as in (2) and compare sha256.
# Classification: parse JSON statements field by
# field; string-scan every payload + second-level
# artifact for _MAX_ZCDP, ZCDP, def/import/
# python/program markers; then recursively decode
# every policy artifact's protobuf wire format and
# rescan all fields, incl. base64/hex decodings
# (scripts e10_wire_decode.py in the artifact).
# 6. Retrieval rate + log scale: time 10 batched
# POST /api/v1/log/entries/retrieve calls (10
# indexes each); log size by binary search over
# checkpoint tree sizes on the same endpoint.
```

Table 1: Condensed audit matrix (27 rows: 4 V, 10 P, 13 T; the denominator is rows, including G4†, which scores no vendor claim—excluding it, 4/10/12 over 26 rows). Row IDs are the stable identifiers of the full matrix (Table 2), where each claim's basis appears in short form; full claim wordings are in Table 8; scores are identical in both tables. Condensation merges A4 into A3 and F2 into B5—both merged rows and both destinations score T, so no aggregation rule affects the totals. V = verified by outsider execution; P = partially checkable (sound in principle, degraded/blocked in practice); T = trust-required (no outsider procedure). The marker * on C5 denotes a negative finding: the executed check contradicted the claim as worded (§3), not a partial success. The marker † on G4 denotes a row that scores no vendor claim: it records the vendor's own reproducibility exclusion, retained as a scope limit on B2 (§5), not a vendor shortfall. The full 29-row matrix is Table 2; per-row hashes, timestamps, and trusted parties are in the artifact package (`AUDIT_MATRIX.md`, which records the matrix as executed—a different 27-row set from this table's: it contains A4 and F2 as separate rows and predates the revision-stage splits, where this condensed table merges A4/F2 away and includes the split rows A2b/C3c); this table is condensed for print.

Dim.	Claim (short)	Score	Anchor
Attestation	**A1** KMS software in public log (exemplar)	V	Rekor idx 1634041292 (virtual) / leaf idx 1512137030; hash-verified; inclusion proof, checkpoint signature, SET, and endorsement signature all verified, 2026-11-04 (amendment experiment)
Attestation	**A2** Published client library implements device-side validation	P	Executed by static inspection of the public client code path (2026-11-04): the library's verifier checks KMS-key + policy endorsements incl. Rekor inclusion proofs
Attestation	**A2b** Production devices run that library	T	Deployment unobservable from public artifacts (§3, rule four)
Attestation	**A3** KMS key release only to matching TEEs	T	Inside TEE, unobservable
Code	**B1** OSS source + public build recipe	V	Repo + run 26261649715, as of 2026-10-08
Code	**B2** Reproducible builds (bit-identical)	P	Project CI invocation executed on a hosted cloud environment, 2026-11-04 (exploratory rescue, §4.3): produced container.tar SHA-256 = endorsed digest d14a1852…(exemplar bundle, single commit); the claim as worded covers all binaries—the rest are unexecuted
Code	**B3** SLSA provenance on Rekor per binary	P	Single-digest scope (20 statements via the GitHub Attestation API; their signatures not independently verified), as of 2026-10-08; the exemplar's own inclusion proof and signatures verified 2026-11-04
Code	**B4** Policy-embedded programs inspectable	P	Policies retrieved in bulk via policy keys (§4.1 census); wire-level decode: complete Python programs embedded (base64) in 5 of 57 distinct artifacts, incl. the Gboard training programs with `_MAX_ZCDP` in code; the rest binary protobufs
Code	**B5** Sideloaded logic stays private	T	By design; characterization trusted
Policy	**C1** Policy pins program + 4-layer digests	P	Mechanism verified in the published proto + Listing-2 worked policy; live Python-workload policy bytes not executed (guide example lapsed, §4.5); inspection-only (the pinning itself not executed on a live policy's bytes)—the evidence class that failed on B4
Policy	**C2** Per-transform access budgets	P	Shown in the guides' printed worked example; not executed on live bytes; inspection-only (the budgets not read in a served policy's bytes)—the evidence class that failed on B4
Policy	**C3** Workloads endorsed under published keys enumerable	P	Enumerated 2026-11-04 (amendment experiment): 509,569 entries over 19 reconstructable keys via the index endpoint; completeness judged on the returned entry set; the claim splits into C3/C3b/C3c (§4.4); two guide fingerprints unqueryable (no key material); payload readability retention-limited
Policy	**C3b** Published key inventory complete	T	No outsider procedure can show that no unpublished endorsement key exists; the inventory is vendor-asserted and already drifted
Policy	**C3c** Index endpoint's returns complete against the log's tree	T	The cross-check authenticates returned entries only; a bounded window reconciliation (2026-11-05 amendment) found no omission in its window (§4.4)
Policy	**C4** 90-day freshness window stated	V	Window stated in ref-values and recomputed, as of 2026-10-08
Policy	**C4b** 90-day freshness window enforced	T	Enforcement inside TEEs, unobservable
Policy	**C5** Historical policies retrievable	P*	≥1-yr retention; 2025 example expired
Keys	**D1** Endorsement keys + rotation public	P	8 endorsement + 1 Rekor key published; fingerprints recomputed; inventory growth observable, rotation not established
Keys	**D2** Key custody/compromise response	T	Log shows use, not custody; the consequence rule applied to a compromise is stated in §5
Keys	**D3** Upload keys bound to policy; TTL expiry	T	Binding code-inspectable, enforcement in-TEE; TTL assurance rests on the design's stated untrusted-time limitation (§5)
Firmware	**E1** Reference values pin FW/SVN	P	Reference values public; full check needs current policy

Table 2: Full audit matrix (29 rows). Condensing it (A4 into A3, F2 into B5) gives the 27-row condensed matrix (Table 1); the modifiers * and † are as defined in Table 1's caption. Scores as scored in this paper; the artifact's `AUDIT_MATRIX.md` records the 27-row matrix as executed, with the evidence; rows A2b and C3c are revision-stage splits under the §3 splitting rule. Rows A1, B2, B3, C3, and G2 carry 2026-11-04 rescorings (rebuild executed but single-instance, §4.3; enumeration split into C3/C3b, §4.4; provenance scoped to its evidence; the disclosure-vs-fact rule of §3). Claim names abbreviated; basis phrases are one-line summaries—full evidence text in the artifact. The marker * on C5 denotes a negative finding: the executed check contradicted the claim as worded (§3), not a partial success. The marker † on G4 denotes a row scoring no vendor claim—a scope limit on B2 (§5), not a vendor shortfall.

Row	Claim (short)	Score	Basis (short)
A1	KMS TEE software recorded in public transparency log (exemplar)	V	Entry, payload, proof, checkpoint + SET + endorsement signatures all verified (amendment experiment)
A2	Published client library validates KMS key + access policy via endorsements + inclusion proofs	P	Static inspection executed 2026-11-04 on the public client library (federated-compute): the attestation verifier checks the upload-key endorsement (KMS keys) and the pipeline configuration (policy keys) via VerifySignedPayload + VerifyRekorLogEntry (inclusion proof recomputed), cross-checking the policy digest in the key endorsement; production devices running it unobservable. P, not V: the executed procedure is static inspection under rule four (§3)—the verifier's code was read, not run; no clean-room reimplementation executed it against a live endorsement
A2b	Production devices run the published client library's validation	T	Deployment unobservable from public artifacts; the library claim is row A2 (§3, rule four)
A3	KMS releases decryption keys only to TEEs matching policy reference values	T	Enforcement inside KMS TEEs; code-inspectable, runtime unobservable (§3 rule)
A4	Root→worker Noise channel with attestation check	T	The check runs inside the TEEs; the quote chain roots in vendor hardware attestation, and no attestation evidence for the channel is published—no outsider-executable validation
B1	KMS + data-processing binaries' source public, with public build recipe	V	Repository and recipe fetched and inspected
B2	Binaries reproducibly buildable (bit-identical)	P	Exemplar bundle rebuilt bit-identically (executed, §4.3; exploratory rescue); other binaries unexecuted
B3	SLSA provenance for every endorsed binary on Rekor	P	Provenance verified on the exemplar digest; every-binary scope unexecuted; statements' signatures not independently verified
B4	Workload Python programs inspectable (embedded in policy)	P	Executed (wire-level decode, 2026-11-04): 5 of 57 distinct (by digest) artifacts embed a complete Python workload program (base64-encoded; 3 distinct texts: Gboard ACT + HMM SentencePiece training, AGSA DP synthesis), each with its `_MAX_ZCDP` guard in code; 52 are binary configurations with no program text in any scanned form (per-artifact record: `records/E10-E11-results.md`). P: the large-key population is sampled, and only served payloads could be decoded (149 of 159 attempts)
B5	Sideloaded proprietary logic stays private	T	Rests on vendor characterization
C1	Policy pins program + binary digests at 4 stack layers	P	Published proto + Listing-2 worked policy verified as printed; live Python-workload policy bytes not executed (guide example lapsed, §4.5); weakest P class—inspection-only: the pinning was read in the published artifacts, not executed against a served policy's bytes (§3, rule four)
C2	Per-transform access budgets (e.g., "at most 1 access")	P	Shown in the guides' printed worked example as printed; not executed on live bytes; same

Table 2: Full audit matrix (29 rows), continued: rows D1–G4.

Row	Claim (short)	Score	Basis (short)
D1	Endorsement keys published; rotation observable	P	Fingerprints recomputed; inventory growth observable; rotation inferred from it, not established
D2	Key custody (who holds the signing keys; compromise response)	T	Custody and response not observable in the log; the consequence rule applied to a compromise is stated in §5
D3	Upload encryption keys bound to policy; TTL enforced	T	Binding code-inspectable, enforcement in-TEE; TTL assurance rests on the design's stated untrusted-time limitation
E1	Reference values pin stage0 firmware digests + SEV-SNP min SVN	P	Reference values public; full check needs current policy
E2	Hardware root of trust (AMD/Intel) intact	T	Hardware roots are vendor trust
F1	Side-channel observability acknowledged as limitation	V	Limitation disclosed in Daly et al.
F2	Sideloaded/proprietary logic protected against malicious operator via future hardware	T	Protection is a future-hardware claim; not testable today
G1	"Only metrics and DP model weights visible to workload operators"	T	Runtime visibility inside the pipeline unobservable
G2	Operator sees which uploads are used per round (§6.3 of Daly et al.)	T	Disclosure verified in §6.3 of Daly et al.; the visibility fact itself is unobservable
G3	DP noise actually added as programmed; budget not exceeded at runtime	T	Noise addition is a runtime act, unobservable
G4	GCP Confidential Space offload containers reproducible	T†	Explicitly carved out of the reproducibility claim in the vendor's docs ("not guaranteed to be reproducible"); retained as a scope limit on B2, not as a vendor claim

Table 3: All-keys census (2026-11-04): entries per published endorsement key via `POST /api/v1/index/retrieve`, and payload verification. "Exhaustive" = every entry's payload fetched; "sampled" = 30 evenly spaced entries (31 for Gboard policy 1, including the guide's example). Two further fingerprints printed in the guide (kernel `6052f352...`, initramfs `5f884b69...`) appear in no current reference-values file and could not be queried (no key material); the guide's own examples for those layers embed the current kernel and initramfs keys (rows `aafd4d5a` and `8546b897`). The entry total counts index returns; payloads were verified per-row as marked in the last column, and across the 430 served payloads there were 0 hash mismatches; the Total row's payload counts sum the attempted retrievals (543 = 430 + 113).

Key (SHA-256 prefix)	Role	Entries	Payloads: matched / unserved
`98fc8ad4`	KMS stage0	51	exhaustive: 27 / 24
`aafd4d5a`	KMS kernel	43	exhaustive: 28 / 15
`8546b897`	KMS initramfs	44	exhaustive: 30 / 14
`0b4fcf07`	KMS kernel cmdline	45	exhaustive: 31 / 14
`a19ea774`	KMS system image	83	exhaustive: 54 / 29
`021205b2`	KMS container key 1	3	exhaustive: 0 / 3
`ea0d1f8f`	Application (container key 2)	115	exhaustive: 110 / 5
`3ebd2cd4`	Gboard policy 1	30,752	sampled: 30 / 1
`49140964`	Gboard policy 2	8	exhaustive: 0 / 8
`e5b8b89b`	Gboard policy 3	0	—
`48877c9b`	Gboard policy 4	0	—
`2c17376f`	Gboard policy 5	0	—
`30cab9ee`	Gboard policy 6	0	—
`e40567d2`	Gboard policy 7	0	—
`927bca59`	Gboard policy 8	0	—
`4d41c79b`	Google App policy	12,051	sampled: 30 / 0
`ee9911ad`	Pixel Search policy	4,607	sampled: 30 / 0
`2bb537ad`	Private Compute Services policy	409,718	sampled: 30 / 0
`5aac231c`	Recorder policy	52,049	sampled: 30 / 0
Total		509,569	430 / 113

Table 4: The three time windows (§4.1, §4.2, §4.5).

Window	Length	Source
Endorsement validity	45 days (difference; 2026-05-26 → 2026-07-10)	Endorsement payload
Freshness window	90 days; integrated after 2025-02-15	Reference values
Store retention	≥1 year	System documentation

Table 5: Per-adversary mapping (from §5 findings).

Adv.	Constrained by	Residual
1	Crypto binding on the audited chain	Substantially constrained
2	Binding + provenance	Runtime unobservable; side channels open
3	Log monitoring only	Defeats the chain for data bound under it; retained history and provenance not rewritable; with operator collusion, the runtime checks too
4	—	Carried as a T residual (hardware roots trusted)
5	Policy digest-pinning	Except through sideloading, whose contents are not logged
6	—	Carried as a T residual (acknowledged, unobservable)

Table 6: Comparator juxtaposition (abbreviated). PCC = Apple Private Cloud Compute (inference); CFC = Google confidential federated compute (Gboard training). The PCC column is documentation-only (a juxtaposition, not a score); the CFC column rests on executed procedures (see §3).

Dimension	CFC (Google)	PCC (Apple)
Transparency log	Rekor (Sigstore/OpenSSF-operated; Google co-created Sigstore [22])	Apple-operated [10]
Source availability	Full except sideloaded workload artifacts, Apache-2.0 (the sideloaded artifacts are not published, §5)	Partial [10]
Reproducible builds	Claimed; one exemplar binary rebuilt bit-identically by us (§4.3)	Explicitly *not* provided (Apple's guide [10]; independent analysis [11])
Policy language	Budgets; embedded programs observed live (5 of 57 in the sampled retrieved set, §4.1)	Attestation→log only [10]
Key custody	Google-held; keys published, rotation not established (§4.2)	Apple-held; not published [10]
Workload	Production training (DP)	*Inference* only [10]
Researcher tooling	Published audit guide	VRE (Virtual Research Environment) + bug bounty [10]; SOC 3 is a program-level audit report, not researcher tooling

Table 7: Composition map: the obligations of the advertised DP guarantee, and this audit's status for each. "Inherited" = taken from cited results, not re-proved here; the DP mathematics itself is out of scope (§1).

Obligation	Established by	Status here
Data bound to the named program and binaries	Policy pinning (C1) + endorsement chain for the exemplar (§4.1)	Pinned in the policy as published (proto + worked example, §4.1); enforcement at upload not observed
Code identity of the endorsed binary	Executed rebuild (§4.3)	Verified (one binary)
Sensitivity and clipping as programmed	Embedded program text in retrieved policies (§4.1)	Inspected, not executed (B4): the `_MAX_ZCDP` guard read in the program text of the five program-carrying artifacts in the retrieved sample (three distinct texts); sensitivity/clipping values rest on that text alone—not demonstrated for policies outside the sample; runtime enforcement not examinable
Noise generation; budget held at runtime	—	Not examinable (G3, T)
Output gating; atomic release with state	—	Not examinable (no separate row; runtime residual, §5)
DP mechanism's mathematics	Daly et al. [1]	Inherited, not re-proved
Leakage channels bounded	Vendor disclosures; Bell-Clark et al. [13]	Partially disclosed (F1)

Table 8: Full claim wordings as scored, with scores. Rows A2b and C3c are revision-stage splits of A2 and C3 under the splitting rule of §3; the remaining wordings are the extracted claim texts of the artifact's `AUDIT_MATRIX.md`.

Row	Claim (full wording)	Score
A1	KMS TEE software recorded in public transparency log (exemplar)	V
A2	The published client library validates the KMS key and the access policy via endorsements and inclusion proofs	P
A2b	Production devices run the published client library's validation	T
A3	KMS releases decryption keys only to TEEs matching policy reference values	T
A4	Root→worker Noise channel with attestation check	T
B1	KMS and data-processing binaries are built from public OSS source, with a public build recipe	V
B2	Binaries reproducibly buildable (bit-identical)	P
B3	SLSA provenance for every endorsed binary on Rekor	P
B4	Workload Python programs inspectable (embedded in policy)	P
B5	Sideloaded proprietary logic stays private	T
C1	Policy pins exact program and root/worker binary digests at four stack layers (firmware/kernel/system/container)	P
C2	Per-transform access budgets (e.g. "at most 1 access")	P
C3	Workloads endorsed under published keys enumerable from the log	P
C3b	Published endorsement-key inventory is complete	T
C3c	The index endpoint's returned entry set for a published key is complete against the log's tree	T
C4	Endorsement freshness window stated (90-day window; not-before 2025-02-15)	V
C4b	Endorsement freshness window enforced	T
C5	Historical policies retrievable (transparency over time)	P*
D1	Endorsement keys published; rotation observable	P
D2	Key custody (who holds the signing keys; compromise response)	T
D3	Upload encryption keys bound to policy; TTL enforced	T
E1	Reference values pin stage0 firmware digests and SEV-SNP minimum SVN	P
E2	Hardware root of trust (AMD/Intel) intact	T
F1	Side-channel observability acknowledged as limitation	V
F2	Sideloaded/proprietary logic protected against a malicious operator via future hardware	T
G1	"Only metrics and DP model weights visible to workload operators"	T
G2	Operator sees which uploads are used per round	T
G3	DP noise actually added as programmed; budget not exceeded at runtime	T
G4	GCP Confidential Space offload containers reproducible	T†

Table 9: Second-scorer exercise (2026-11-04): agreement on the 23 condensed rows of the superseded constitution, as then constituted—21 of 23, Cohen's $\kappa = 0.85$ (§3). Both disagreements scored our published score too generous—the enumeration row (P vs. T, on the evidence then presented) and the per-round upload-use row (P vs. T)—and both were adopted at the time. The exercise predates the C3/C4 splits and this revision's A2 and C3c splits; its 23 rows are the published condensed matrix (27 rows, less the four rows absorbed by the three merges) with the C3 family, the C4 pair, and the A2 pair each carried as one merged row; it records the superseded constitution's scoring, not the current inventory's.

	Second: V	Second: P	Second: T	Published total
Published V	2	0	0	2
Published P	0	11	2	13
Published T	0	0	8	8
Second total	2	11	10	23

Table 10: The §5 variant cells, tabulated. Counts are V/P/T over the row total in the last column, computed under the merge rule and fixed computation order stated in §5 (re-scorings first, then merges, then removals; a re-scoring and a removal of the same row are alternatives, never stacked). The joint envelope over the unmarked cells—the scorer axis crossed in its strict direction only—is V 3–5, P 6–11, coordinatewise. Cells marked † are amendment-state cells (a reverted score): they record superseded scoring states, are not envelope members, and are the only cells in which the verified count goes below 3.

Variant	V/P/T	Rows
Published condensed matrix	4/10/13	27
Published full matrix	4/10/15	29
C3 family merged back	4/10/11	25
C4 pair merged	3/11/12	26
G2 scored as its disclosure	5/10/12	27
Strict C1/C2 line (C1, C2 at T)	4/8/15	27
Weakest-link C3 merge	4/9/12	25
Weakest-link C4 merge	3/10/13	26
Weakest-link merges, both	3/9/12	24
Remove G4	4/10/12	26
Remove G4 and F2 (full matrix)	4/10/13	27
B2 flip (P→T) × strict C1/C2 line	4/7/16	27
A3 at P × C4 merge	3/12/11	26
B2 at V	5/9/13	27
G4 at P	4/11/12	27
Stacked strict readings (C1/C2 line, B2 flip, weakest-link C3)	4/6/15	25
Stacked strict readings + weakest-link C4	3/6/15	24
Snapshot (A1, C3 reverted)†	3/10/12	25
Snapshot × G2-disclosure variant†	4/10/11	25
Snapshot × strict line†	3/8/14	25
Snapshot × C4 merge†	2/11/11	24
Snapshot × weakest-link C4 merge†	2/10/12	24
A1 reverted alone × C4 merge†	2/12/12	26